In [1]:
```
#import Libaries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import sqlite3
```

In [2]:
```
#import Database
```

In [3]:
```
conn = sqlite3.connect('customer_churn.db')
sql_query = """ SELECT name
                FROM sqlite_master
                WHERE type='table'
                """
tables = pd.read_sql(sql_query,conn)

#create Dataframe
for table_name in tables['name']:
    df=pd.read_sql(f"SELECT * FROM {table_name}",conn)
    globals()[f"df_{table_name}"] = df
    print(f"Created dataframe: df_{table_name}")

conn.close()
```

```
Created dataframe: df_db_customer
Created dataframe: df_db_subscription
Created dataframe: df_db_support
```

In [4]:
```
#print table name and column name
conn = sqlite3.connect('customer_churn.db')
for table_name in tables['name']:
    print(f"\nTable name:{table_name}")
    #get column information
    columns_query= f"PRAGMA table_info({table_name});"
    columns=pd.read_sql(columns_query,conn)
    print("Columns:")
    print(columns['name'].tolist())

conn.close()
```

```
Table name:db_customer
Columns:
['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincod
e']

Table name:db_subscription
Columns:
['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'p
lan_type', 'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_
charges', 'cltv', 'churn_score']

Table name:db_support
Columns:
['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

In [5]:
```
# Data Cleaning
```

In [6]:
```
df_db_customer.head()
```

Out[6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None

In [7]:
```
df_db_customer.tail()
```

Out[7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [8]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 8 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   customerid  21 non-null     str
 1   name        21 non-null     str
 2   country     18 non-null     str
 3   state       21 non-null     str
 4   gender      21 non-null     str
 5   dob         21 non-null     str
 6   interests   4 non-null      str
 7   pincode     0 non-null      object
dtypes: object(1), str(7)
memory usage: 1.4+ KB
```

In [9]:
```
# error needs to fix in Data
#1>rename col-name
#2>drop columns-interest and pincode
#3>change data type-dob
#4>fix missing values -country
```

In [10]:
```
df_db_customer.rename(columns = {'name' : 'customer_name'},inplace=True)
df_db_customer
```

Out[10]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> <th>interests</th> <th>pincode</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00	travel	None
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00	NaN	None
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00	movie	None
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00	NaN	None
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00	drama	None
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00	NaN	None
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00	NaN	None
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00	NaN	None
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00	NaN	None
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00	NaN	None
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00	NaN	None
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00	NaN	None
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00	NaN	None
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00	NaN	None
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00	NaN	None
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00	job	None
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00	NaN	None

0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00	NaN	None
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00	NaN	None
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00	NaN	None
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00	NaN	None

In [11]:
```
# df_db_customer.drop(df_db_customer.columns[-2:],axis=1,inplace=True)
df_db_customer.drop(columns=['interests','pincode'],inplace=True)
```

In [12]:
```
df_db_customer
```

Out[12]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

0002-ORFBO	keshav	India	Maharashtra	Male	1982-04-12 00:00:00
0003-MKNFE	raghav	India	Karnataka	Male	1995-11-23 00:00:00
0004-TLHLJ	lalita	India	Delhi	Female	1978-02-15 00:00:00
0011-IGKFF	mohan	India	Nagaland	Male	2001-08-30 00:00:00
0013-EXCHZ	mira	India	Delhi	Female	1990-05-05 00:00:00
0013-MHZWF	durga	NaN	Delhi	Women	1988-12-10 00:00:00
0013-SMEOE	mina	India	Meghalaya	Female	1976-09-21 00:00:00
0014-BMAQU	madan	India	Rajasthan	Male	1999-03-14 00:00:00
0015-UOCOJ	maya	NaN	Kathmandu	Women	1985-07-07 00:00:00
0016-QLJIS	arjun	Nepal	Kathmandu	Male	1993-10-29 00:00:00
0017-DINOC	shiva	India	Maharashtra	Men	1997-01-22 00:00:00
0017-IUDMW	rangadevi	India	Karnataka	Female	1981-06-18 00:00:00
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01 00:00:00
0019-EFAEP	raju	India	Meghalaya	Female	1992-04-25 00:00:00
0019-GFNTW	Madhav	India	Uttar Pradesh	Men	1979-11-11 00:00:00
0020-INWCK	parvati	India	Delhi	Female	1986-02-28 00:00:00
0020-JDNXP	rikim	India	Meghalaya	Female	1994-08-19 00:00:00
0021-IKXGC	vishakha	India	Rajasthan	Female	2000-09-02 00:00:00
0022-TCJCI	raghvendra	India	Telangana	Male	1983-12-30 00:00:00
0023-HGHWL	rishabh	India	Uttar Pradesh	Men	1991-05-14 00:00:00
0023-UYUPN	sudevi	India	Maharashtra	Women	1977-10-06 00:00:00

In [13]:
```
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     str
dtypes: str(6)
memory usage: 1.1 KB
```

In [14]:
```
df_db_customer['dob'] = pd.to_datetime(df_db_customer['dob'])
df_db_customer.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   customerid     21 non-null     str
 1   customer_name  21 non-null     str
 2   country        18 non-null     str
 3   state          21 non-null     str
 4   gender         21 non-null     str
 5   dob            21 non-null     datetime64[us]
dtypes: datetime64[us](1), str(5)
memory usage: 1.1 KB
```

In [15]:
```
df_db_customer['gender'].unique()
```

Out[15]:
```
<StringArray>
['Male', 'Female', 'Women', 'Men']
Length: 4, dtype: str
```

In [16]:
```
df_db_customer['gender']=df_db_customer['gender'].replace({'Men':'Male','Women':
df_db_customer['gender'].unique()
```

Out[16]:
```
<StringArray>
['Male', 'Female']
Length: 2, dtype: str
```

In [17]:
```
#handeling Missing values
df_db_customer[df_db_customer['country'].isna()]
```

Out[17]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> <th>5</th> <th>8</th> <th>12</th> </tbody> <tbody></tbody>

0013-MHZWF	durga	NaN	Delhi	Female	1988-12-10
0015-UOCOJ	maya	NaN	Kathmandu	Female	1985-07-07
0018-NYROU	chitra	NaN	Telangana	Female	2004-12-01

In [18]:
```
#df_db_customer['country'].fillna("NA")
df_db_customer[['country','state']]
```

Out[18]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>country</th> <th>state</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> <th>9</th> <th>10</th> <th>11</th> <th>12</th> <th>13</th> <th>14</th> <th>15</th> <th>16</th> <th>17</th> <th>18</th> <th>19</th> <th>20</th> </tbody> <tbody></tbody>

India	Maharashtra
India	Karnataka
India	Delhi
India	Nagaland
India	Delhi
NaN	Delhi
India	Meghalaya
India	Rajasthan
NaN	Kathmandu
Nepal	Kathmandu
India	Maharashtra
India	Karnataka
NaN	Telangana
India	Meghalaya
India	Uttar Pradesh
India	Delhi
India	Meghalaya
India	Rajasthan
India	Telangana
India	Uttar Pradesh
India	Maharashtra

In [19]:
```
#counrty and state - Unique value pair
state_country_mapping=df_db_customer.dropna(subset=['country']).set_index('state
df_db_customer['country']=df_db_customer['country'].fillna(df_db_customer['state
```

In [20]:
```
df_db_customer[df_db_customer['country'].isna()]
```

Out[20]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>customer_name</th> <th>country</th> <th>state</th> <th>gender</th> <th>dob</th> </thead> <tbody> </tbody> <tbody></tbody>

In [ ]:

In [21]:
```
df_db_subscription.head()
```

Out[21]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	churn_score
0	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaN	NaN	13.99	627	12
1	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
2	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaN	NaN	6.99	210	34
3	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaN	NaN	22.99	1725	8
4	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88

In [22]:
```
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype  
---  ------                  --------------  -----  
 0   customerid              21 non-null     str    
 1   subscription_start_date 21 non-null     str    
 2   subscription_type       21 non-null     str    
 3   renewal_date            21 non-null     str    
 4   plan_type               21 non-null     str    
 5   contract_type           21 non-null     str    
 6   cancellation_date       6 non-null      str    
 7   cancellation_reason     6 non-null      str    
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64  
 10  churn_score             21 non-null     int64  
dtypes: float64(1), int64(2), str(8)
memory usage: 1.9 KB
```

In [23]:
```
date_col=['subscription_start_date','renewal_date','cancellation_date']
df_db_subscription[date_col]=df_db_subscription[date_col].apply(pd.to_datetime)
df_db_subscription.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 21 entries, 0 to 20
Data columns (total 11 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   customerid              21 non-null     str
 1   subscription_start_date 21 non-null     datetime64[us]
 2   subscription_type       21 non-null     str
 3   renewal_date            21 non-null     datetime64[us]
 4   plan_type               21 non-null     str
 5   contract_type           21 non-null     str
 6   cancellation_date       6 non-null      datetime64[us]
 7   cancellation_reason     6 non-null      str
 8   monthly_charges         21 non-null     float64
 9   cltv                    21 non-null     int64
 10  churn_score             21 non-null     int64
dtypes: datetime64[us](3), float64(1), int64(2), str(5)
memory usage: 1.9 KB
```

In [24]:
```
df_db_support.head()
```

Out[24]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>col_1</th> <th>comment</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody> </tbody>

0003-MKNFE	2024-08-28 00:00:00	N	60	None	service issue
0003-MKNFE	2024-08-28 00:00:00	Y	10	None	demaned refund
0013-EXCHZ	2024-01-20 00:00:00	Y	20	None	NaN
0013-MHZWF	2025-03-18 00:00:00	N	90	None	guidance to renew
0013-SMEOE	2024-11-01 00:00:00	N	30	None	NaN

In [25]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
 4   col_1           0 non-null      object
 5   comment         4 non-null      str
dtypes: int64(1), object(1), str(4)
memory usage: 564.0+ bytes
```

In [26]:
```
df_db_support.drop(columns=['col_1','comment'],inplace=True)
```

In [27]:
```
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      str
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: int64(1), str(3)
memory usage: 420.0 bytes
```

In [28]:
```
df_db_support['complaint_date']= pd.to_datetime(df_db_support['complaint_date'])
df_db_support.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 9 entries, 0 to 8
Data columns (total 4 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   customerid      9 non-null      str
 1   complaint_date  9 non-null      datetime64[us]
 2   escalations     9 non-null      str
 3   csat_score      9 non-null      int64
dtypes: datetime64[us](1), int64(1), str(2)
memory usage: 420.0 bytes
```

In [ ]:
```

```

In [29]:
```
#Feature engineering and Data analysis
```

In [30]:
```
#create new column using existing column
df_db_subscription['churn_flag']=np.where(df_db_subscription['cancellation_date'
```

In [31]:
```
df_db_subscription.head()
```

Out[31]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>subscription_start_date</th> <th>subscription_type</th> <th>renewal_date</th> <th>plan_type</th> <th>contract_type</th> <th>cancellation_date</th> <th>cancellation_reason</th> <th>monthly_charges</th> <th>cltv</th> <th>churn_score</th> <th>churn_flag</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual	NaT	NaN	13.99	627	12	0
0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91	1
0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly	NaT	NaN	6.99	210	34	0
0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual	NaT	NaN	22.99	1725	8	0
0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88	1

In [32]:
```
#first fix support table duplicates then merge
df=(df_db_subscription.merge(df_db_customer,on ='customerid',how='left')
                .merge(df_db_support, on='customerid',how='left'))
```

In [33]:
```
df.shape
```

Out[33]: (23, 20)

In [34]:
```
df_db_subscription['customerid'].unique()
```

Out[34]:
```
<StringArray>
['0002-ORFBO', '0003-MKNFE', '0004-TLHLJ', '0011-IGKFF', '0013-EXCHZ',
 '0013-MHZWF', '0013-SMEOE', '0014-BMAQU', '0015-UOCOJ', '0016-QLJIS',
 '0017-DINOC', '0017-IUDMW', '0018-NYROU', '0019-EFAEP', '0019-GFNTW',
 '0020-INWCK', '0020-JDNXP', '0021-IKXGC', '0022-TCJCI', '0023-HGHWL',
 '0023-UYUPN']
Length: 21, dtype: str
```

In [35]:
```
df_db_customer['customerid'].unique()
```

Out[35]:
```
<StringArray>
['0002-ORFBO', '0003-MKNFE', '0004-TLHLJ', '0011-IGKFF', '0013-EXCHZ',
 '0013-MHZWF', '0013-SMEOE', '0014-BMAQU', '0015-UOCOJ', '0016-QLJIS',
 '0017-DINOC', '0017-IUDMW', '0018-NYROU', '0019-EFAEP', '0019-GFNTW',
 '0020-INWCK', '0020-JDNXP', '0021-IKXGC', '0022-TCJCI', '0023-HGHWL',
 '0023-UYUPN']
Length: 21, dtype: str
```

In [36]:
```
df_db_support['customerid'].unique()
```

Out[36]:
```
<StringArray>
['0003-MKNFE', '0013-EXCHZ', '0013-MHZWF', '0013-SMEOE', '0017-IUDMW',
 '0019-EFAEP', '0022-TCJCI']
Length: 7, dtype: str
```

In [37]:
```
df_db_support['complaint_count']=df_db_support.groupby('customerid')['customerid
```

In [38]:
```
df_db_support
```

Out[38]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>customerid</th> <th>complaint_date</th> <th>escalations</th> <th>csat_score</th> <th>complaint_count</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> <th>5</th> <th>6</th> <th>7</th> <th>8</th> </tbody> <tbody></tbody>

0003-MKNFE	2024-08-28	N	60	2	
0003-MKNFE	2024-08-28	Y	10	2	
0013-EXCHZ	2024-01-20	Y	20	1	
0013-MHZWF	2025-03-18	N	90	1	
0013-SMEOE	2024-11-01	N	30	1	
0017-IUDMW	2024-04-10	Y	25	1	
0019-EFAEP	2024-09-27	Y	30	1	
0022-TCJCI	2024-09-13	Y	10	2	
0022-TCJCI	2024-09-14	N	90	2	

In [39]:
```
df_db_support=df_db_support.sort_values('complaint_date').drop_duplicates('custo
```

In [40]:
```
df=(df_db_subscription.merge(df_db_customer,on ='customerid',how='left')
                .merge(df_db_support, on='customerid',how='left'))
```

In [41]:
```
df.shape
```

Out[41]:
```
(21, 21)
```

In [42]:
```
df.to_csv('exported_churn_data.csv',index=False)
```

In [43]:
```
#Data Analysis
```

In [44]:
```
df.columns
```

Out[44]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count'],
      dtype='str')
```

In [45]:
```
#Churn Rate
```

In [46]:
```
churn_rate=df['churn_flag'].mean()*100
print("Churn Rate =",round(churn_rate,2),"%")
```

```
Churn Rate = 28.57 %
```

In [47]:
```
# Retention Rate
retention_rate=100- churn_rate
print("Retention rate=",round(retention_rate,2),"%")
```

```
Retention rate= 71.43 %
```

In [48]:
```
#churn by Plan Type
churn_by_plan=(df.groupby('plan_type')['churn_flag'].mean().mul(100).round(2).re
print(churn_by_plan)
```

```
  plan_type  churn_rate_pct
0     Basic           60.00
1   Premium           14.29
2  Standard           22.22
```

In [49]:
```
# churn by state + sum(revenue) & count of users
```

In [50]:
```
# ARPU - Avg Revenue per user
arpu=df['monthly_charges'].mean()
print("ARPU = ",round(arpu,2))
```

```
ARPU =  18.85
```

In [51]:
```
#Average Customer Tenure
# count of days users has used our service : cancellation date else current date
today=pd.Timestamp.today()
df['tenure_days']= np.where(
     df['cancellation_date'].notna(),
    (df['cancellation_date']-df['subscription_start_date']).dt.days,
    (today-df['subscription_start_date']).dt.days

)
avg_tenure = df['tenure_days'].mean()
print("Average Tenure (days)=",round(avg_tenure),0)
```

```
Average Tenure (days)= 1537 0
```

In [52]:
```
# Revenue at risk - revenue lost from churned users
revenue_at_risk=df.loc[df['churn_flag']==1,'monthly_charges'].sum()
print("Revenue at Risk (Rs Lakhs)=",revenue_at_risk)
```

```
Revenue at Risk (Rs Lakhs)= 73.94
```

In [53]:
```
# Esclation Rate
escalation_rate = (df['escalations']=='Y').mean()*100
print("Escalation Rate = ",round(escalation_rate,2),"%")
```

```
Escalation Rate =  19.05 %
```

In [54]:
```
# AVG Complaint Per User
avg_complaints=df['complaint_count'].sum()/df['customerid'].nunique()
print("Avg Complaints Per User=",round(avg_complaints,2))
```

```
Avg Complaints Per User= 0.43
```

In [55]:
```
# Correlation Escalation vs Churn
```

```
df['escalations']=np.where(df['escalations']=='Y',1,0)  #Encoding String to int
corr_df=df[['escalations','churn_flag']].dropna()

correlation=corr_df['escalations'].corr(df['churn_flag'])
print("Correlation between escalation vs churn is =",round(correlation,2))
```

```
Correlation between escalation vs churn is = 0.77
```

In [56]:
```
# churn risk- Create a new column using existing column
conditions= [
    (df['churn_score']<50),
    (df['churn_score']>=50) & (df['churn_score']<70),
    (df['churn_score']>=70)
]
choices = ['low','med','high']

df['churn_risk']=np.select(conditions,choices,default='unkown')
```

In [ ]:
```

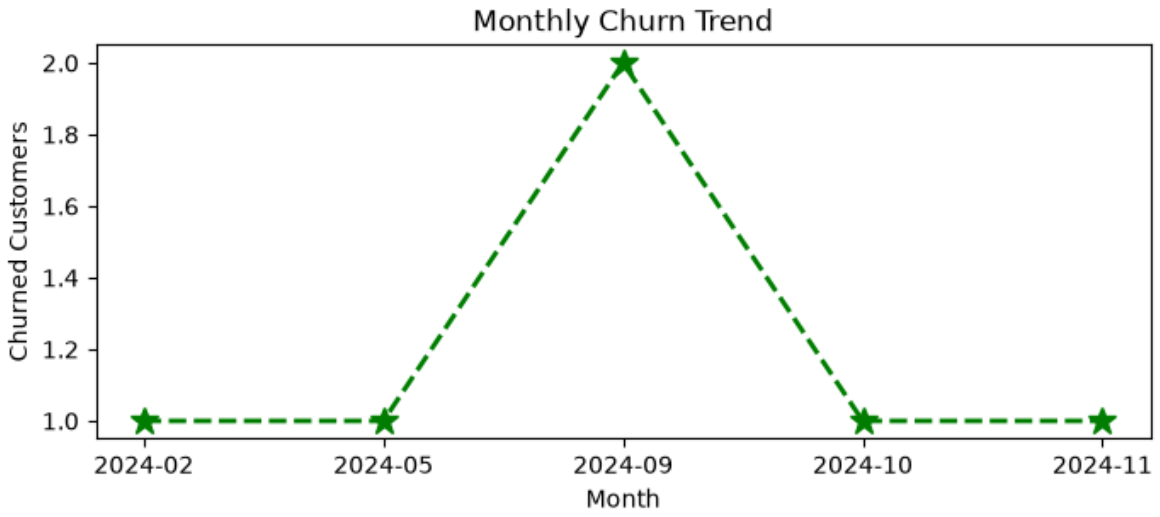
```

In [57]:
```
#Visualization
```

In [58]:
```
df_visual=df.copy()
```

In [59]:
```
# Monthly churn Trends(Time series KPI)

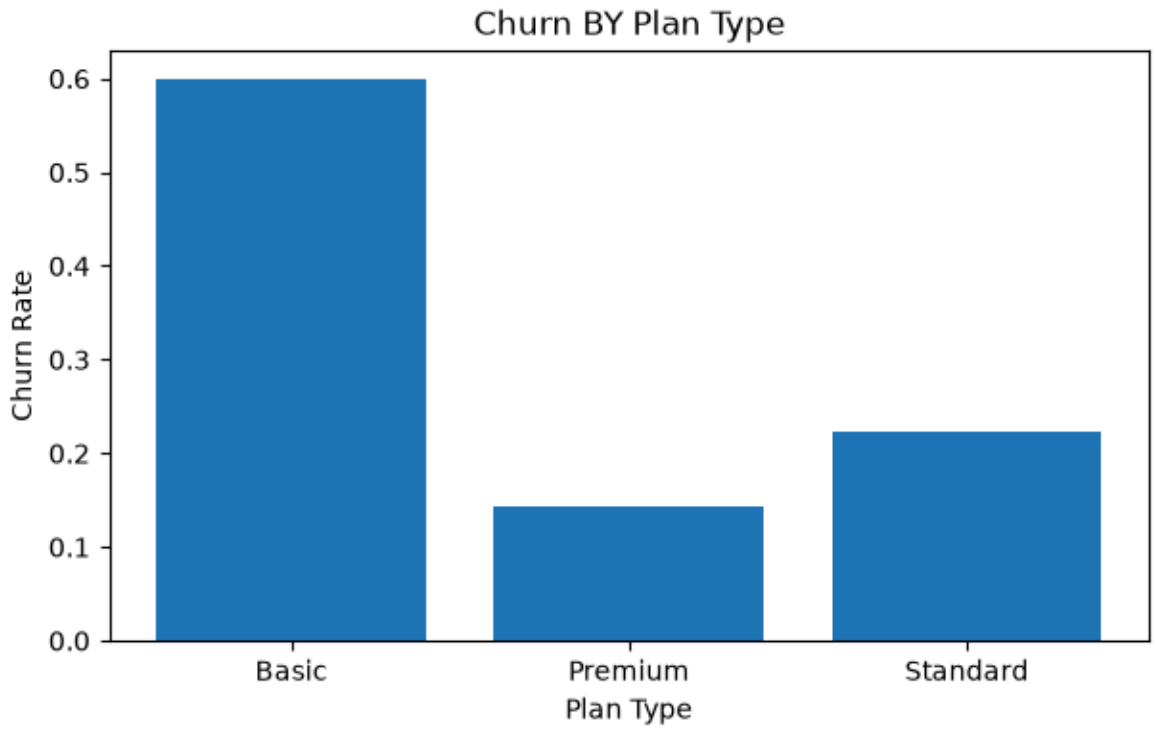
df_visual['cancellation_month']=df_visual['cancellation_date'].dt.to_period("M")
churn_trend=df_visual[df_visual['churn_flag']==1].groupby('cancellation_month').
plt.figure(figsize=(8,3))
plt.plot(churn_trend.index.astype(str),churn_trend.values,color='green',marker='
plt.title("Monthly Churn Trend")
plt.xlabel("Month")
plt.ylabel("Churned Customers")
plt.show()
```



In [60]:
```
# Churn By Plan Type

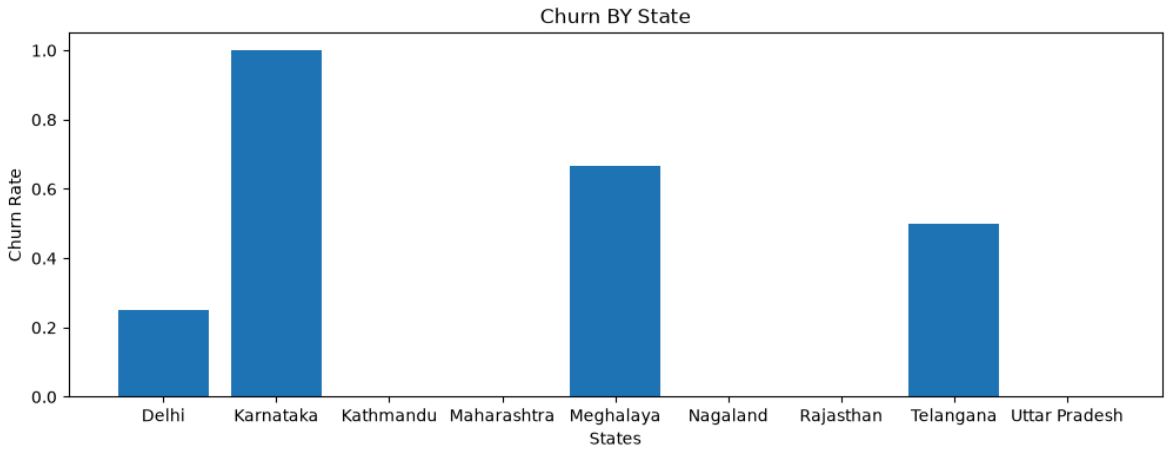
churn_plan=df_visual.groupby('plan_type')['churn_flag'].mean()
plt.figure(figsize=(7,4))
plt.title("Churn BY Plan Type")
plt.bar(churn_plan.index,churn_plan.values)
plt.xlabel("Plan Type")
```

```
plt.ylabel("Churn Rate")
plt.show()
```



In [61]:
```
# Churn By States

churn_plan=df_visual.groupby('state')['churn_flag'].mean()
plt.figure(figsize=(12,4))
plt.bar(churn_plan.index,churn_plan.values)
plt.title("Churn BY State")
plt.xlabel("States")
plt.ylabel("Churn Rate")
plt.show()
```



In [ ]:
```

```

In [62]:
```
#encoding ;-converting str to numeric so that we can find correlation between fe
df_visual.columns
```

Out[62]:
```
Index(['customerid', 'subscription_start_date', 'subscription_type',
       'renewal_date', 'plan_type', 'contract_type', 'cancellation_date',
       'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score',
       'churn_flag', 'customer_name', 'country', 'state', 'gender', 'dob',
       'complaint_date', 'escalations', 'csat_score', 'complaint_count',
       'tenure_days', 'churn_risk', 'cancellation_month'],
      dtype='str')
```

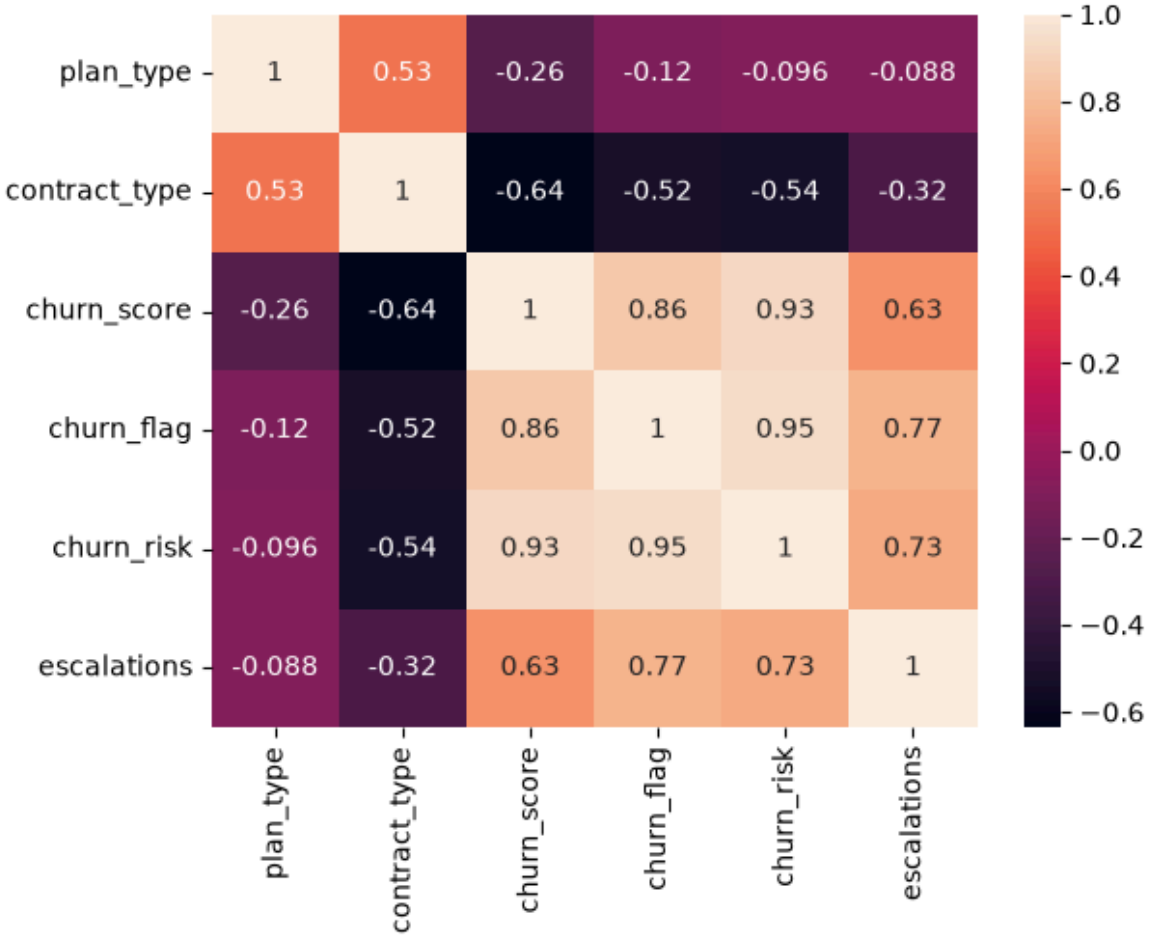
In [63]:
```
df_encoded=df_visual[[
       'plan_type', 'contract_type', 'churn_score',
       'churn_flag','churn_risk', 'escalations']]
order_mappings={
    'plan_type':['Basic','standard','Premium'],
    'contract_type':['Monthly','Annual'],
    'churn_risk':['low','med','high']}

for col, order in order_mappings.items():
    df_encoded[col]=pd.Categorical(df_encoded[col].astype('category'), categorie
```

```
C:\Users\ACER\AppData\Local\Temp\ipykernel_11292\3733943583.py:10: Pandas4Warnin
g: Constructing a Categorical with a dtype and values containing non-null entries
not in that dtype's categories is deprecated and will raise in a future version.
  df_encoded[col]=pd.Categorical(df_encoded[col].astype('category'), categories=o
rder,ordered=True).codes
```

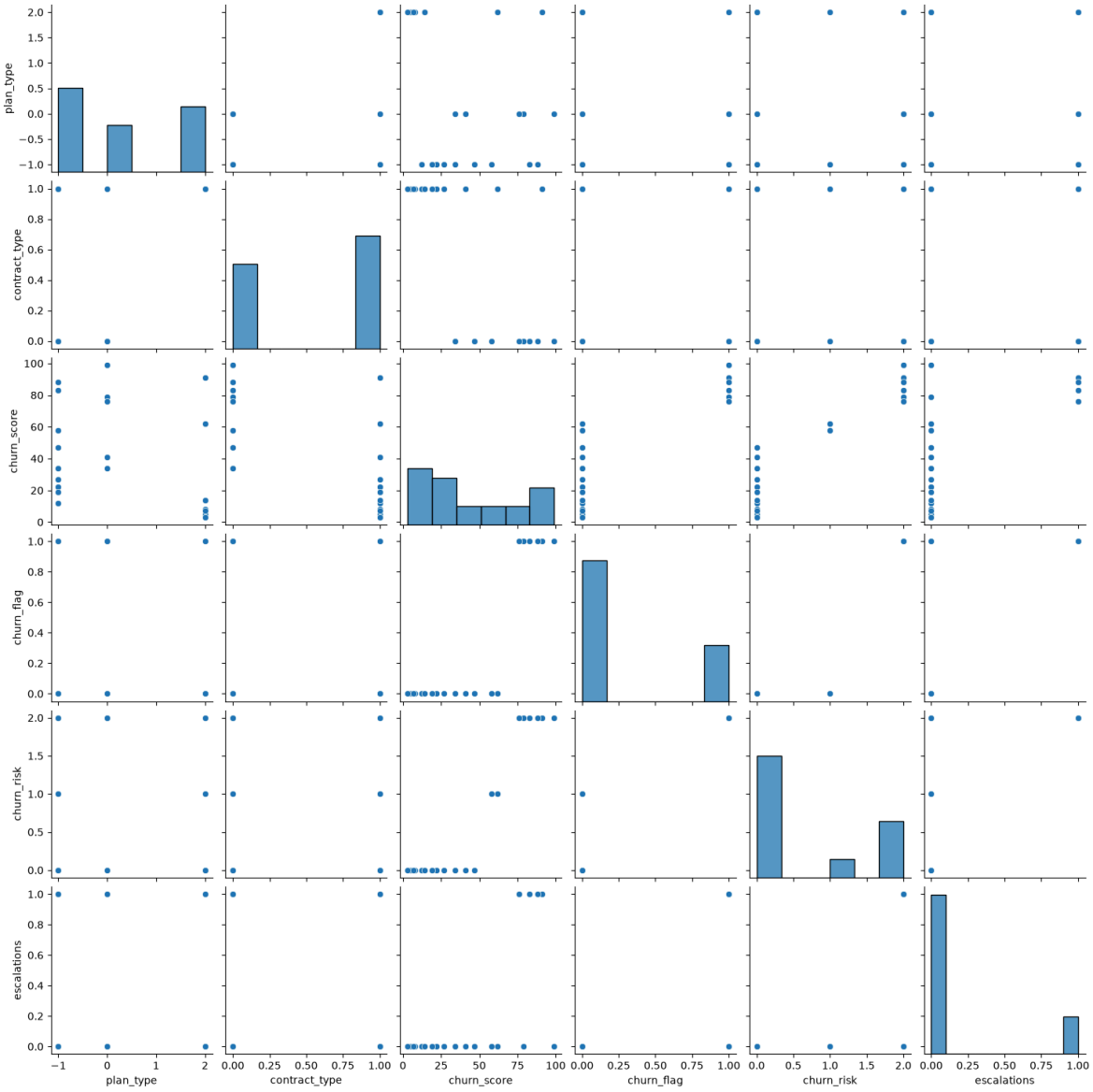
In [64]:
```
# Heatmap(correlation_matrix)
sns.heatmap(df_encoded.corr(), annot=True)
```

Out[64]: <Axes: >



In [65]:
```
# pairplot :-relationship in a dataset
sns.pairplot(df_encoded)
```

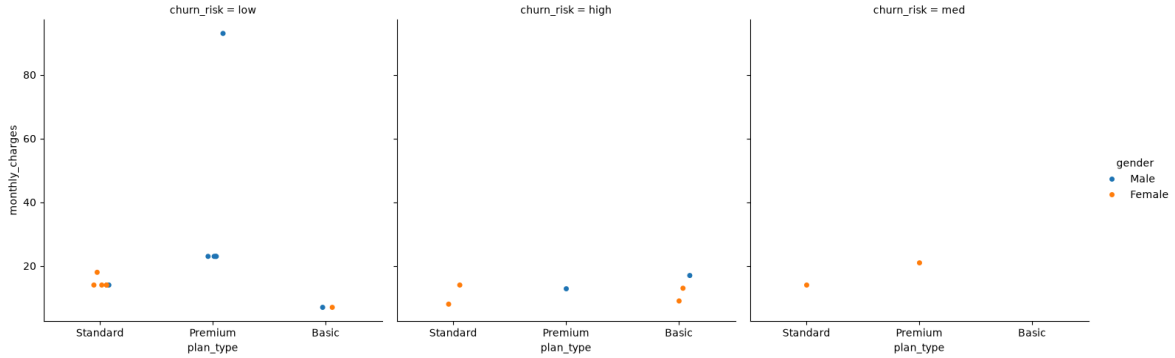
Out[65]: <seaborn.axisgrid.PairGrid at 0x1e4488e02f0>



In [66]:
```
# catplot/Facegrid plot
sns.catplot(data=df_visual,
            x='plan_type',
            y='monthly_charges',
            hue='gender',

            col='churn_risk')
```

Out[66]: <seaborn.axisgrid.FacetGrid at 0x1e44d5b6ba0>



In [67]:
```
# pivot Table
```

In [68]:
```
pd.pivot_table(
  df_visual,
    values=['monthly_charges','customerid','churn_flag'],
    index='plan_type',
    aggfunc={
        'monthly_charges':'sum'
        ,'customerid': 'nunique',
        'churn_flag':'mean'
    }

).reset_index()
```

Out[68]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>plan_type</th> <th>churn_flag</th> <th>customerid</th> <th>monthly_charges</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> </tbody> <tbody></tbody>

Basic	0.600000	5	52.95
Premium	0.142857	7	218.93
Standard	0.222222	9	123.91

In [ ]:

In [ ]: